



LECTURE 0 — THE COURSE AND ROS 2

AUTONOMOUS ROBOTIC SYSTEMS ENGINEERING

Topics: Course content · Labs · Grading · ROS 2 · Lab 0

TODAY

- **Part 1** — the course: content, labs, grading
- **Part 2** — ROS 2: concepts, tools, a real robot
- **Lab 0** — what to do before the next session

Everything Lab 0 needs is in the second half.



PART 1 · THE COURSE

WHAT YOU BUILD, AND HOW IT IS GRADED



COURSE CONTENT

TOPIC	
ROS 2 architecture	nodes, topics, packages, launch
Modelling and simulation	Gazebo, the robot model, worlds
Sensors and time	measurement models, noise, synchronisation
State estimation	Kalman and particle filters, sensor fusion
Computer vision	features, matching, optical flow, visual odometry
SLAM	maps, pose graphs, loop closure
Planning and navigation	costmaps, global and local planning
Control and autonomy	from a plan to wheel commands — the full pipeline

 docs.ros.org/en/jazzy/Concepts.html



LABS

- Bring-up — environment, a robot that drives, first measurements
- SLAM and navigation with off-the-shelf packages
- **Your own robot model**
- **Your own odometry**
- **Your own optical flow**
- **Your own sensor fusion**
- **Your own planner plugin**
- **Your own stack, end to end**

Each lab builds on the one before it: what you produce in the first is the input to the second.

HOW THE LABS MOVE

	YOU USE	YOU WRITE
First	an off-the-shelf robot and packages	the measuring side
Then	one layer at a time	that layer
Last	your own stack	all of it

- The same world, the same robot, the same metric throughout
- Every result is comparable with the off-the-shelf package — and with your own previous one

One layer per lab, until the stack is yours.

REQUIREMENTS

You need — Python and/or C++ · Linux and the command line · git

I give you

- a Docker image: ROS 2 Jazzy + Gazebo Harmonic, versions pinned
- data with ground truth, and reference numbers to check your own against
- a starting point for every lab

Week 1: an entry survey. What you already know decides how much arrives ready-made.

 docs.ros.org/en/jazzy/Installation.html

WHAT EACH LAB SHIPS

A **good README** is the deliverable. It contains:

- **Run** — the exact commands, from zero, on someone else's machine
- **See** — screenshot, recording, plot
- **Numbers** — the metric of that lab
- **Compare** — your result against the off-the-shelf package doing the same job

Plus `notes.txt` — what failed, which ideas you had, what stayed unclear. Your own writing, and it cannot lower your mark.

In a year this repository is read by an employer, not by me.

GENERATIVE AI

- **Allowed, for understanding** — explain someone's code, unpick an error, find the documentation
- **Self-restraint on the labs** — generated code gives a result without the understanding
- No black boxes in your own stack
- Checking is built into the course, not into a detector: the interview, the numbers, your own code

A simple implementation you can explain beats a complex one you cannot.

GRADING

COMPONENT	WEIGHT
Labs	50
Talk	20
Exam	20
Open-source contribution	10

Three of the four stay with you after the course: the repositories, the talk, the PR.

TALK – 15-20 MIN

Your topic, agreed with me. What works:

- a gap in this course — something we skipped or rushed
- differences between ROS 2 releases: Humble → Jazzy → what comes next
- a package worth reading — what is inside, where its limits are
- an algorithm against its implementation: what the authors simplified
- a robot or platform — its architecture, not its marketing

The same weight as the exam — prepare it like one. The gaps you wrote into `notes.txt` are your best source of a topic.

OPEN SOURCE · EXAM

Open source — one real project, one real issue.

- pick a project you actually use; find an issue you understand
- propose your ideas **in the issue, in public**; say why this project and this issue
- pick a repo next to your own stack — Nav2, `slam_toolbox`, `robot_localization`, `gz-sim`; filter `good first issue`
- the merge is **not** graded — the analysis, the change and the tests are
- why it is here: you get inside a real codebase, and it is a fast way to learn

Exam — 15–20 min. Your results and numbers · your decisions and where they stop working · what goes on your CV.

DEADLINES

- Every lab has a deadline, fixed in writing in the first week
- After it, the mark is lower
- **The reference solution is published before the deadline** — check yourself against it; what you hand in is still your own work
- A lab that never lands blocks the next one — the labs feed each other

PART 2 · ROS 2

THE MIDDLEWARE YOU WILL LIVE IN

 docs.ros.org/en/jazzy/Concepts/Basic.html



WHAT ROS IS

- **Middleware** — how separate programs exchange data
- **Build system** — `colcon` · `ament` · `rosdep`
- **Tooling** — `ros2` CLI, RViz, rqt, rosbag
- **Ecosystem** — Nav2, `slam_toolbox`, drivers, simulators

The valuable part is not the middleware — it is the shared interfaces.

Any lidar publishes the same message, so someone else's SLAM runs on your robot.

⚠ `roscore` · `roslaunch` · `catkin` in a tutorial → that is ROS 1, EOL since May 2025.

- One release for all of us: Jazzy Jalisco on Ubuntu 24.04, pinned in the container

 docs.ros.org/en/jazzy/Concepts/Basic.html · design.ros2.org/articles/changes.html

WHY NOT ONE BIG PROGRAM

ONE PROGRAM	A GRAPH OF NODES
the caller names the callee: <code>slam.update(scan)</code>	the driver names a topic: <code>/scan</code>
one language, one rate, one machine	Python and C++, 40 Hz and 2 Hz, laptop and robot
replace a part → recompile the whole thing	replace a part → start a different node
you get a return value	you get no answer at all

- So `/scan` can come from the lidar, from the simulator or from a recording, and the reader never notices
- The price: no reply, no acknowledgement, and a copy of every message

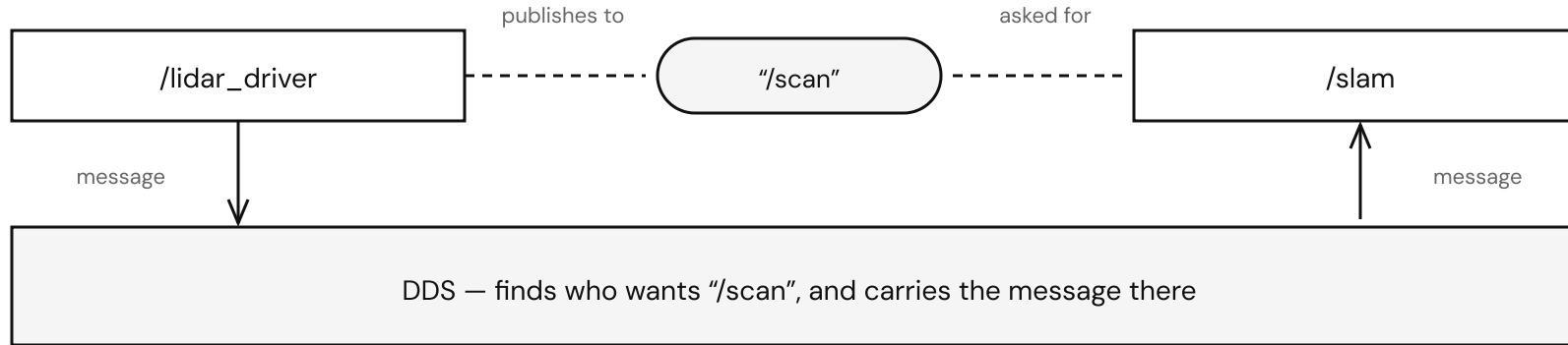
You give up the function call, and buy a system whose parts can be swapped while it runs.

NODES

- A process that does one job — lidar driver, odometry, planner
- Has a name in the graph; a namespace separates two copies: `/robot1/odometry`
- A node knows topic names, not other nodes — kill one and the rest survives
- Two nodes may carry the same name — `ros2 node list` warns, nothing crashes; the cure is a namespace

```
$ ros2 run demo_nodes_cpp talker  
$ ros2 node list  
/talker  
/listener
```

DDS – WHAT MOVES THE MESSAGES



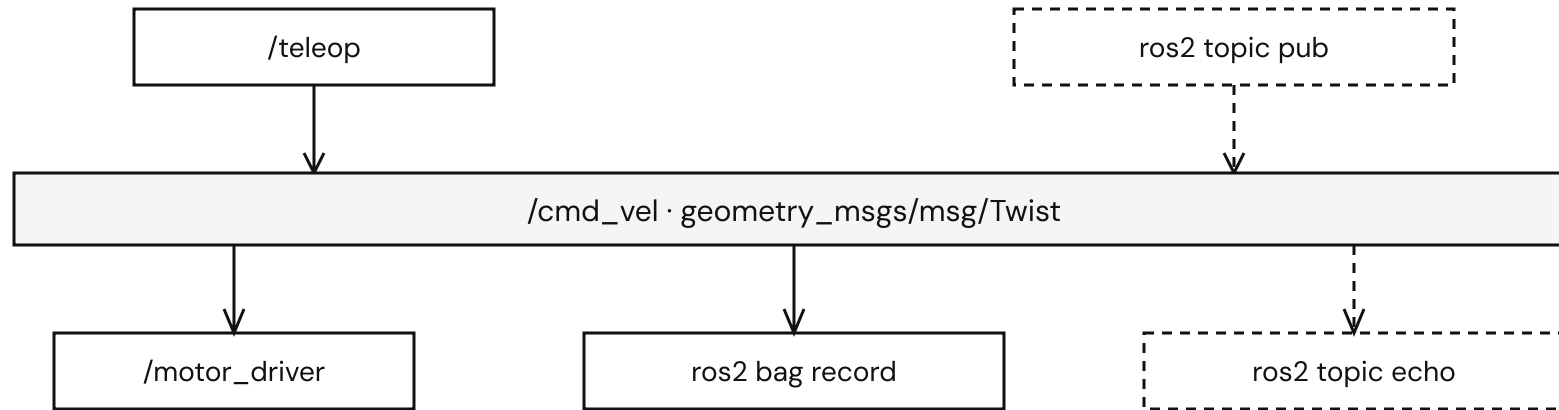
Data Distribution Service — a standard for sending typed messages between programs.

- A topic is a **name**. Something still has to carry the bytes to whoever wants them — that is DDS
- You say “publish `/scan`”; DDS delivers it to every node that asked for `/scan`
- That is why a topic needs no address: you name the **data**, not a destination
- The same on one machine and across a network — ROS 2 reuses this standard instead of writing its own

design.ros2.org/articles/ros_on_dds.html

TOPICS

90 % of the semester. A named, typed channel — nobody addresses anybody.



No arrow goes box to box — everything goes through the name.

- `publish()` does not block and returns nothing: no acknowledgement, and no error when nobody listens
- A finite queue sits between `publish()` and the callback — fall behind and messages are dropped silently

**WHY DOES ROS 2 NEED A MIDDLEWARE – WHY NOT LET
EVERY NODE OPEN A SOCKET TO THE NODE IT NEEDS?**

 design.ros2.org/articles/ros_on_dds.html



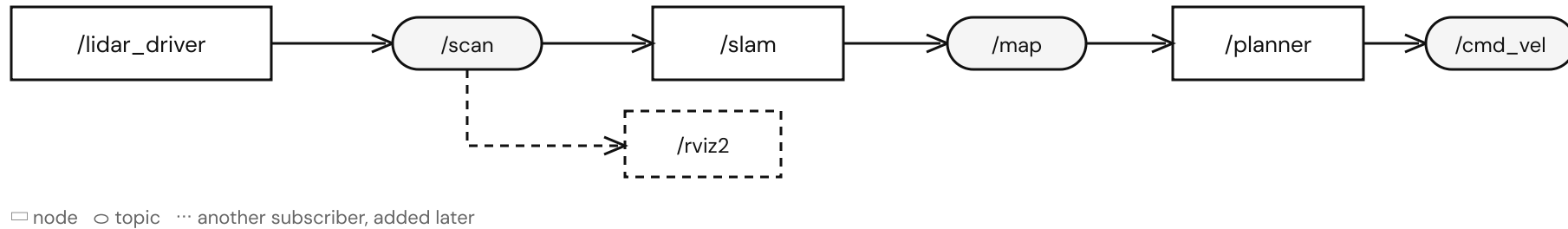
ANSWER

- **Nobody would know the address.** Nodes come and go; a socket needs a host and a port before it can connect
- **It is not one-to-one.** A scan goes to the mapper, the logger and the viewer at once — sockets would mean N connections to maintain
- **Someone has to agree on the bytes.** The middleware carries typed messages, so both sides read the same fields
- What you would end up writing, if you did it yourself, is a worse DDS

 design.ros2.org/articles/ros_on_dds.html



THE GRAPH



- Assembled at runtime — it is not written down in one file
- **Running half the system is normal.** It simply has less data
- `ros2 node info` · `rqt_graph` show what actually connected
- Start `rviz2` mid-run and it just connects — the publisher never learns about it
- Kill a node and only its edges vanish — the rest of the system keeps running

RVIZ – LOOKING AT THE DATA

- A 3D viewer for live ROS data: the laser scan, the camera image, the map, the robot's own shape, the path it plans
- **It is not a simulator.** It draws what is on the topics; it does not compute anything and it does not move the robot
- It is an ordinary node — it subscribes like any other, and shows nothing if nobody publishes
- This is where you find out whether the data is wrong or your algorithm is: if the scan looks wrong in RViz, stop debugging the algorithm

 docs.ros.org/en/jazzy/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html



**YOU START THE SUBSCRIBER BEFORE ANY PUBLISHER
EXISTS. WHAT HAPPENS?**

 docs.ros.org/en/jazzy/Concepts/Intermediate/About-Discovery.html



ANSWER

It waits — they connect when the publisher appears.

- Start order does not matter; `ros2 launch` may bring nodes up in any order
- **No error if the publisher never appears at all** — the two cases look identical
- Which is why you check with `ros2 topic info` — not by reading the log

 docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html

MESSAGE TYPES – SENSORS

SOURCE	TYPE	TOPIC
2D lidar	<code>sensor_msgs/msg/LaserScan</code>	<code>/scan</code>
3D lidar, depth	<code>sensor_msgs/msg/PointCloud2</code>	<code>/points</code>
Camera	<code>sensor_msgs/msg/Image</code> + <code>CameraInfo</code>	<code>/camera/image_raw</code>
IMU	<code>sensor_msgs/msg/Imu</code>	<code>/imu/data</code>
GNSS	<code>sensor_msgs/msg/NavSatFix</code>	<code>/gps/fix</code>
Wheels, joints	<code>sensor_msgs/msg/JointState</code>	<code>/joint_states</code>

- Every one of these carries a `Header` — a stamp and a frame; without it nothing can be fused
- `Image` alone is useless for geometry: `CameraInfo` carries the intrinsics

 docs.ros.org/en/jazzy/p/sensor_msgs

MESSAGE TYPES – WHAT THE STACK COMPUTES

MEANING	TYPE	TOPIC
Where the robot is	<code>nav_msgs/msg/Odometry</code>	<code>/odom</code>
The map it built	<code>nav_msgs/msg/OccupancyGrid</code>	<code>/map</code>
Where to drive	<code>geometry_msgs/msg/Twist</code>	<code>/cmd_vel</code>

The topic names are conventions, not rules — but every third-party package expects them.

- A package you did not write already knows what `/odom` and `/map` mean
- That is the whole reason someone else's navigation can drive your robot

THE TYPE IS THE CONTRACT

```
$ ros2 interface show sensor_msgs/msg/LaserScan
std_msgs/Header header      # when, and in which frame
float32 angle_min           # start angle of the scan [rad]
float32 angle_increment      # angular distance between measurements [rad]
float32[] ranges             # the measurements [m]
```

`/scan` can come from Gazebo · a real lidar driver · `ros2 bag play` · your own test node. **Your code does not change.**

A custom type isolates you: no third-party package can read it. Invent one only when no standard type fits.

WHAT THE STANDARD TYPES BUY

- Publish `sensor_msgs/msg/LaserScan` → RViz draws it, `ros2 bag` records it, `slam_toolbox` maps with it
- Publish `nav_msgs/msg/Odometry` and subscribe to `geometry_msgs/msg/Twist` → Nav2 can drive your robot
- Add `sensor_msgs/msg/Imu` → `robot_localization` fuses it with your odometry
- None of those packages know your robot exists. You registered nothing and wrote no adapter
- Over 2 000 packages are released for Jazzy — one `apt install` each

Your robot joins the ecosystem by publishing the right message. That is the entire entry fee.

WHAT YOU GET FOR FREE

PACKAGE	CONSUMES → PRODUCES
Nav2	a goal pose → <code>/cmd_vel</code> : costmaps, planner, controller, recovery
slam_toolbox	<code>/scan</code> → a map and a pose, with loop closure
ros2_control	a body twist → wheel commands; encoders → odometry
robot_localization	wheel odometry + IMU + GNSS → one fused pose
image_pipeline	a raw camera stream → calibrated, rectified, depth
MoveIt 2	the same job for arms: inverse kinematics, planning, collision checking

Already installed with it: RViz · Gazebo and the `ros_gz` bridge · `rosbag2` · `teleop_twist_keyboard`

 index.ros.org · docs.nav2.org



ONE CONTROLLER, MANY ROBOTS

- A **controller** is a plugin, not a node you write: `diff_drive_controller` turns a body twist into left and right wheel velocities, and integrates the encoders back into odometry
- It drives *any* differential-drive base because it never sees your motors — only a wheel joint and its velocity. Its parameters are the wheel names, the wheel separation and the wheel radius
- You provide the hardware side: which joints exist, and code that reads encoders and writes commands
- In simulation that hardware side is provided for you — same description, same controller settings
- **“Every robot?”** Every differential-drive base. Another kinematics, another controller: ackermann, mecanum, omni — and a robot without wheel encoders gets no odometry at all

We meet it again as soon as we model our own robot: describe the wheels, pick the controller.

LOOKING AT THE DATA

```
$ ros2 topic echo /scan --once      # one message, printed
$ ros2 topic hz /scan
average rate: 9.984                 # how often it really arrives
$ ros2 topic bw /camera/image_raw
27.31 MB/s                          # what a camera actually costs you
$ ros2 topic info /scan --verbose
Publisher count: 1                  # who is on the other end
Subscription count: 2
```

There is no file that says how the system is wired. There is only the live graph — and these commands.

- `list -t` first — the name is usually right and the type is not

 docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html

`ros2 topic list` SHOWS `/scan`.

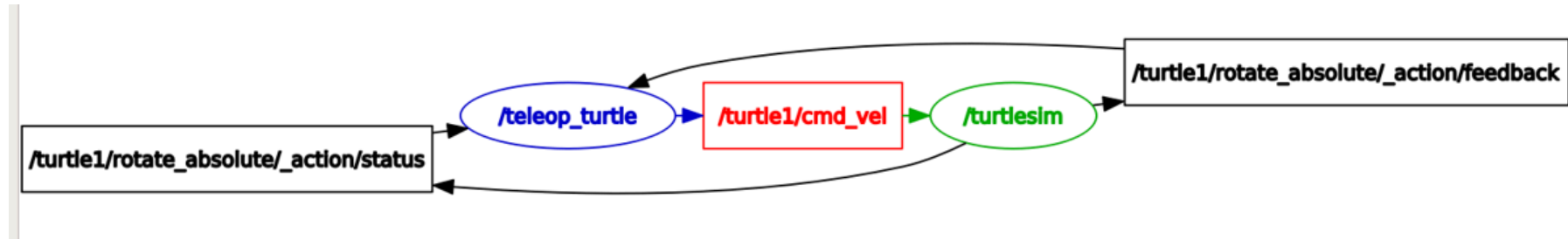
`ros2 topic echo /scan` PRINTS NOTHING, AND NO
ERROR APPEARS ANYWHERE. WHAT IS WRONG?

ANSWER

The topic exists, but nothing is publishing to it.

- `ros2 topic list` shows a name as soon as *anyone* mentions it — a subscriber alone is enough
- `ros2 topic info /scan --verbose` prints the publisher count. Zero publishers, no data, no error
- The other usual cause is a typo: you are echoing `/scan` while the driver publishes `/scan_raw`
- Silence is a normal state in ROS 2 — that is why you check the graph, not the log

RQT_GRAPH – THE GRAPH, DRAWN



Ovals are nodes, rectangles are topics. Screenshot: ROS 2 documentation, CC BY 4.0

- One command — `rqt_graph` — and you see who is actually connected to whom
- The fastest way to discover that two things you believed were talking are not

PUBLISHING FROM THE TERMINAL

```
$ ros2 topic pub -r 10 /cmd_vel geometry_msgs/msg/Twist \  
  "{linear: {x: 0.2}, angular: {z: 0.5}}"
```

The robot drives an arc. No code written.

- The receiving node cannot tell a terminal from a planner — same topic, same type
- So any node can be tested by standing in for its missing neighbour
- “Should work, but the robot is not moving” → the first thing you do is `echo`
- `--once` sends a single message and exits; without `-r` the default is 1 Hz
- A typo in the topic name is not an error — you publish to a brand-new topic nobody reads

 docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Topics/Understanding-ROS2-Topics.html



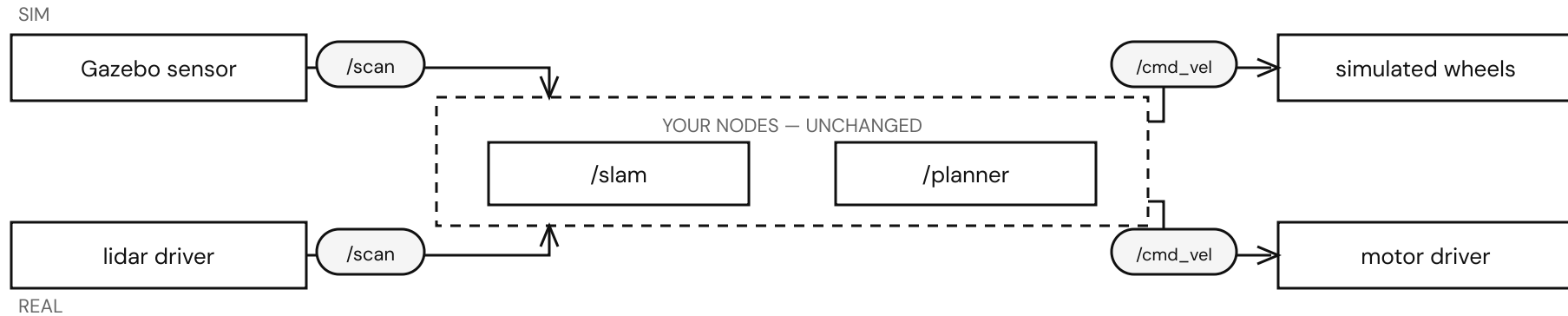
RECORDING WHAT HAPPENED

```
$ ros2 bag record --topics /scan /odom /cmd_vel    # or -a for everything
$ ros2 bag info rosbag2_2026_09_07-14_22_10       # topics, duration, message count
$ ros2 bag play rosbag2_2026_09_07-14_22_10      # replay it as live data
```

- A subscriber cannot tell a bag from a live sensor — same topic, same type
- So an algorithm is debugged without a robot, deterministically, in CI
- You and I measure on **identical data** — a bag ships with every lab

 docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Recording-And-Playing-Back-Data/Recording-And-Playing-Back-Data.html

FROM SIMULATION TO A REAL ROBOT



Swap the ends of the graph. Two things change in your configuration:

- **use_sim_time** — in simulation every node takes its clock from the simulator; on the robot it is the wall clock again
- **Ground truth disappears** — the simulator knew exactly where the robot was; reality does not
- A real driver publishes best-effort sensor data — a reliable subscription connects to nothing

A REAL ROBOT – TURTLEBOT 3 BURGER

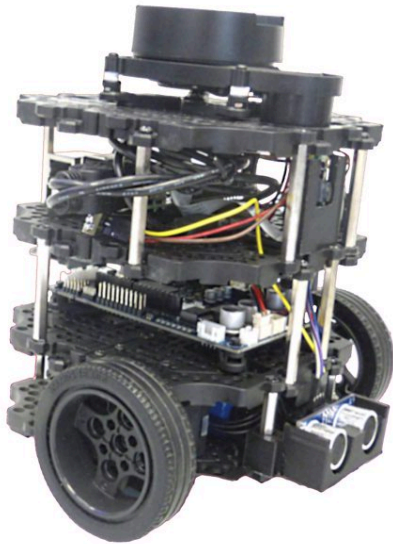


Photo: KuscuO, Wikimedia Commons, CC BY-SA 4.0

Two computers. A Raspberry Pi runs ROS 2; a small control board runs the motors.

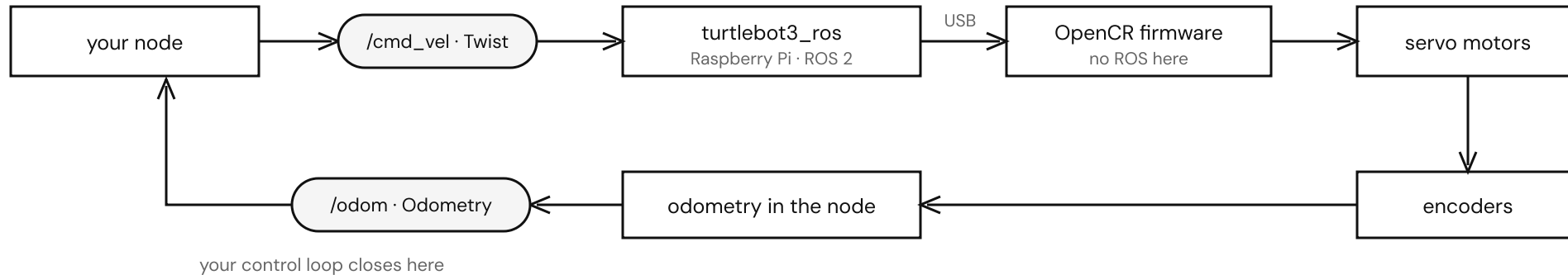
Why two? Linux is good at graphs and networking, and bad at hitting a deadline. Driving motors is a small loop that must run on time, every time — so it gets its own board.

Sensors. 360° laser scanner · IMU on the control board · encoders inside the two servo motors · battery monitor.

It publishes: `/scan` · `/odom` · `/imu` · `/joint_states` · `/battery_state`

It subscribes to one motion topic: `/cmd_vel`

FROM A **Twist** MESSAGE TO TURNING WHEELS



⚠ In simulation the velocity you ask for is the velocity you get. On the robot there is firmware you did not write, a cable in between, and wheels that slip.

SERVICES AND ACTIONS

	WHAT IT IS	WHEN
Topic	stream, no reply	sensor data, commands
Service	request → response	a quick one-off call
Action	goal → feedback → result	a long task
Parameter	a named setting of a node	change behaviour, no rebuild

- Longer than a second, or you may need to cancel it → action
- `ros2 service list -t` · `ros2 action list -t` — the same introspection you already use on topics
- Nav2 hands you actions; from the second lab on you send goals to them from your own node

 docs.ros.org/en/jazzy/Concepts/Basic/About-Services.html · docs.ros.org/en/jazzy/Concepts/Basic/About-Actions.html

A PACKAGE – THE SIMPLEST ONE

```
$ ros2 pkg create --build-type ament_python my_pkg --dependencies roslpy sensor_msgs
```

One command, and you get:

```
ws/src/my_pkg/  
├─ package.xml      # name, license, dependencies – this is what makes it a ROS package  
├─ setup.py         # what to install, and which script becomes a runnable node  
└─ my_pkg/  
    └─ my_node.py   # your code
```

- A package is the unit you build, install, share and depend on — never a loose script
- `package.xml` is why `rosdep install` can fetch your dependencies on someone else's machine
- `build/`, `install/`, `log/` are generated — they never go into git

 docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html



PYTHON OR CMAKE?

BUILD TYPE	YOU WRITE	USE IT FOR
<code>ament_python</code>	<code>setup.py</code>	Python nodes — everything in Lab 0
<code>ament_cmake</code>	<code>CMakeLists.txt</code>	C++ nodes

- Both produce the same thing: a package `colcon` builds and `ros2 run` can start
- The same workspace holds both, and the two talk over the same topics

 docs.ros.org/en/jazzy/How-To-Guides/Ament-CMake-Documentation.html



PARAMETERS

```
$ ros2 param list /my_node  
$ ros2 param get /my_node speed  
$ ros2 param set /my_node speed 0.5
```

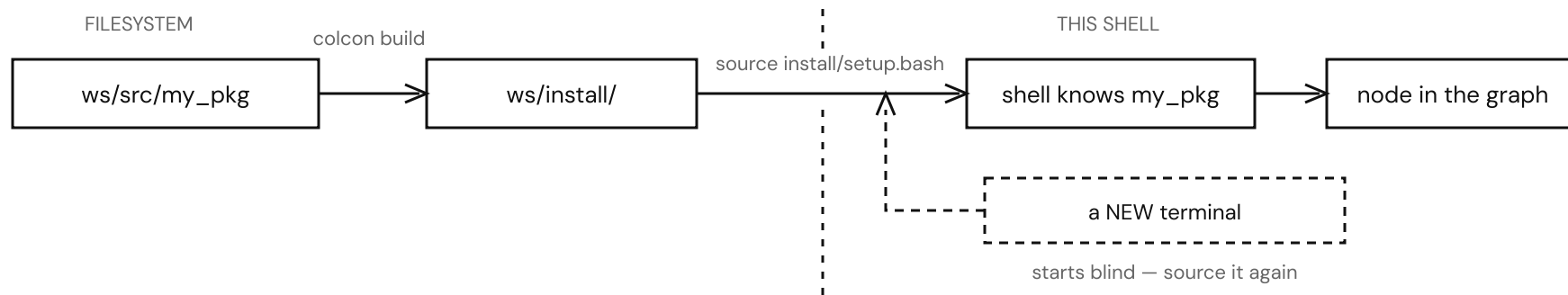
- Settings live outside the code: a threshold, a topic name, a gain, a frame rate
- The same node, different robot, different run — no rebuild
- Set from the command line, from a launch file, or from a YAML file
- A parameter has to be declared by the node before it can be set

 docs.ros.org/en/jazzy/Concepts/Basic/About-Parameters.html



BUILD AND RUN

```
$ colcon build --packages-select my_pkg --symlink-install  
$ source install/setup.bash  
$ ros2 run my_pkg my_node
```



docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Colcon-Tutorial.html

**CHANGE ONE LINE OF CODE AND YOU MUST REBUILD AND
RE-SOURCE. CHANGE A NUMBER IN A YAML FILE AND
YOU NEED NEITHER. WHY?**

ANSWER

Code is installed; configuration is read.

- `colcon build` copies (or links) your package into `install/`, and `ros2 run` starts what is there — not what is in `src/`
- `source install/setup.bash` is what tells this shell where `install/` is
- A YAML file is read by the node **at startup**, so a new value only needs a restart
- Which is why anything you expect to tune — a speed limit, a threshold, a topic name — belongs in a parameter file, not in the code

LAUNCH FILES – WHAT ACTUALLY STARTS

```
$ ros2 launch my_robot bringup.launch.py
```

Three terminals' worth of nodes, from one file:

NODE IT STARTS	WHAT THAT NODE DOES
robot_state_publisher	publishes the robot's shape and joint positions
lidar_driver	reads the sensor, publishes <code>/scan</code>
diff_drive_controller	turns <code>/cmd_vel</code> into wheel commands

- One command instead of three terminals, in a fixed order, with fixed parameters
- Topic renaming and parameters live here, not in the code — same system, different setup, no rebuild
- `--show-args` lists everything you can override from the command line

 docs.ros.org/en/jazzy/Tutorials/Intermediate/Launch/Launch-Main.html



COMMANDS FOR LAB 0

```
ros2 run <pkg> <node>           # start a node
ros2 node list · ros2 node info  # who is in the graph
ros2 topic list -t               # what is being published
ros2 topic echo · hz · info      # look at the data
ros2 topic pub                   # publish by hand
rqt_graph                        # the graph, drawn
colcon build --packages-select    # build one package
source install/setup.bash        # make it visible to ros2
```

 docs.ros.org/en/jazzy/Tutorials.html



SETTING IT UP

1. Ubuntu 24.04 — `sudo apt install ros-jazzy-desktop`
2. `source /opt/ros/jazzy/setup.bash` in every new terminal, or once in `.bashrc`
3. Make a workspace — `mkdir -p ~/ws/src`, then run `colcon build` from `~/ws`
4. `source ~/ws/install/setup.bash` — now `ros2 run` finds your own packages
5. Check it: `ros2 run demo_nodes_cpp talker` here, `ros2 run demo_nodes_py listener` there

Same version for all of us: Jazzy on Ubuntu 24.04, or the course container on any other OS.

LAB 0

Not graded. No git, no README.

1. **Install ROS 2 Jazzy** — the five steps from the previous slide. Build from source is optional

2. Create a workspace and a package

3. One node publishing a sensor-style message, one subscribing to it —

`Range` · `Imu` · `Temperature` · `BatteryState` · `LaserScan`

4. Check it: `ros2 topic list` · `echo` · `hz`

- Use a sensor type, not `std_msgs/String` — the rest of the course runs on `sensor_msgs`
- The two links below are the whole lab: install, then workspace → package → publisher → subscriber

 docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html · docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries.html



WHAT'S NEXT

Lecture 1 — Gazebo and ROS 2

Gazebo: physics, plugins, worlds and sensors · models from Fuel · running without a window · the ROS 2 ↔ Gazebo bridge · RViz · simulation time

- Before then: `ros2 launch simple_bot bot.launch.py` — a robot that already drives from `/cmd_vel`